

Tutorial t-SNE en R: buenas prácticas

Jesús F García Gavilán

1. Paquetes:

Rtsne: implementación eficiente de t-SNE (con aproximación Barnes-Hut) ggplot2: visualización

```
paquetes = c("Rtsne", "ggplot2")
nuevos = paquetes[!(paquetes %in% installed.packages()[, "Package"])]
if (length(nuevos)) install.packages(nuevos)

library(Rtsne)
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

2. Reproducibilidad

La semilla se fija ANTES de cualquier paso estocástico (t-SNE inicializa las posiciones de forma aleatoria, así que esto es imprescindible)

```
set.seed(123)
```

3. Datos

Simulamos un dataset con estructura de clústeres conocida, para poder comprobar después si t-SNE recupera esa estructura. En un caso real, esto sería vuestra matriz de muestras x variables (genes, metabolitos, biomarcadores...)

```
n_por_grupo = 100
n_variables = 50

grupo_a = matrix(rnorm(n_por_grupo * n_variables, mean = 0),
                 nrow = n_por_grupo)
grupo_b = matrix(rnorm(n_por_grupo * n_variables, mean = 3),
                 nrow = n_por_grupo)
grupo_c = matrix(rnorm(n_por_grupo * n_variables, mean = -3, sd = 2),
                 nrow = n_por_grupo)

datos = rbind(grupo_a, grupo_b, grupo_c)
etiquetas = factor(rep(c("Grupo A", "Grupo B", "Grupo C"),
                      each = n_por_grupo))
```

4. Preprocesado

4.1 Eliminar duplicados exactos (Rtsne falla si los encuentra)

```
duplicados = duplicated(datos)
if (any(duplicados)) {
  datos = datos[!duplicados, ]
  etiquetas = etiquetas[!duplicados]
}
```

4.2 Escalar los datos ANTES de pasarlos a Rtsne, como paso independiente

Esto evita confundir “varianza alta porque la variable está en otra escala” con “varianza alta porque hay señal real”

```
datos_escalados = scale(datos)
```

5. Elegir la perplejidad

Regla práctica: la perplejidad debe ser mucho menor que el número de muestras (como mucho un tercio, normalmente bastante menos). Con $n = 300$ muestras, probamos varios valores para ver el efecto

```
perplejidades = c(5, 30, 50)

resultados = list()
for (p in perplejidades) {
  resultados[[as.character(p)]] = Rtsne(X = datos_escalados,
    dims = 2,
    perplexity = p,
    theta = 0.5,
    pca = TRUE,
    pca_center = TRUE,
    pca_scale = FALSE,
    check_duplicates = FALSE,
    max_iter = 1000,
    verbose = TRUE)
}
```

```
## Performing PCA
## Read the 300 x 50 data matrix successfully!
## Using no_dims = 2, perplexity = 5.000000, and theta = 0.500000
## Computing input similarities...
## Building tree...
## Done in 0.00 seconds (sparsity = 0.073800)!
## Learning embedding...
## Iteration 50: error is 75.674515 (50 iterations in 0.01 seconds)
## Iteration 100: error is 73.811829 (50 iterations in 0.01 seconds)
## Iteration 150: error is 75.981116 (50 iterations in 0.01 seconds)
## Iteration 200: error is 76.927896 (50 iterations in 0.01 seconds)
```

```

## Iteration 250: error is 78.375097 (50 iterations in 0.01 seconds)
## Iteration 300: error is 2.381083 (50 iterations in 0.01 seconds)
## Iteration 350: error is 1.722731 (50 iterations in 0.01 seconds)
## Iteration 400: error is 1.493023 (50 iterations in 0.01 seconds)
## Iteration 450: error is 1.434579 (50 iterations in 0.01 seconds)
## Iteration 500: error is 1.402132 (50 iterations in 0.01 seconds)
## Iteration 550: error is 1.376090 (50 iterations in 0.01 seconds)
## Iteration 600: error is 1.361445 (50 iterations in 0.01 seconds)
## Iteration 650: error is 1.354101 (50 iterations in 0.01 seconds)
## Iteration 700: error is 1.344098 (50 iterations in 0.01 seconds)
## Iteration 750: error is 1.337130 (50 iterations in 0.01 seconds)
## Iteration 800: error is 1.329436 (50 iterations in 0.01 seconds)
## Iteration 850: error is 1.323205 (50 iterations in 0.01 seconds)
## Iteration 900: error is 1.318791 (50 iterations in 0.01 seconds)
## Iteration 950: error is 1.314273 (50 iterations in 0.01 seconds)
## Iteration 1000: error is 1.311920 (50 iterations in 0.01 seconds)
## Fitting performed in 0.15 seconds.
## Performing PCA
## Read the 300 x 50 data matrix successfully!
## Using no_dims = 2, perplexity = 30.000000, and theta = 0.500000
## Computing input similarities...
## Building tree...
## Done in 0.01 seconds (sparsity = 0.319911)!
## Learning embedding...
## Iteration 50: error is 47.917260 (50 iterations in 0.01 seconds)
## Iteration 100: error is 46.910648 (50 iterations in 0.01 seconds)
## Iteration 150: error is 47.832441 (50 iterations in 0.01 seconds)
## Iteration 200: error is 48.507156 (50 iterations in 0.01 seconds)
## Iteration 250: error is 49.867485 (50 iterations in 0.01 seconds)
## Iteration 300: error is 1.165228 (50 iterations in 0.01 seconds)
## Iteration 350: error is 1.016803 (50 iterations in 0.01 seconds)
## Iteration 400: error is 0.995679 (50 iterations in 0.01 seconds)
## Iteration 450: error is 0.984586 (50 iterations in 0.01 seconds)
## Iteration 500: error is 0.973754 (50 iterations in 0.01 seconds)
## Iteration 550: error is 0.956250 (50 iterations in 0.01 seconds)
## Iteration 600: error is 0.850455 (50 iterations in 0.01 seconds)
## Iteration 650: error is 0.772203 (50 iterations in 0.01 seconds)
## Iteration 700: error is 0.770443 (50 iterations in 0.01 seconds)
## Iteration 750: error is 0.769005 (50 iterations in 0.01 seconds)
## Iteration 800: error is 0.767741 (50 iterations in 0.01 seconds)
## Iteration 850: error is 0.767666 (50 iterations in 0.01 seconds)
## Iteration 900: error is 0.766427 (50 iterations in 0.01 seconds)
## Iteration 950: error is 0.766113 (50 iterations in 0.01 seconds)
## Iteration 1000: error is 0.765479 (50 iterations in 0.01 seconds)
## Fitting performed in 0.22 seconds.
## Performing PCA
## Read the 300 x 50 data matrix successfully!
## Using no_dims = 2, perplexity = 50.000000, and theta = 0.500000
##Computing input similarities...
## Building tree...
## Done in 0.01 seconds (sparsity = 0.593867)!
## Learning embedding...
## Iteration 50: error is 41.727961 (50 iterations in 0.01 seconds)
## Iteration 100: error is 40.388593 (50 iterations in 0.01 seconds)

```

```
## Iteration 150: error is 39.204929 (50 iterations in 0.01 seconds)
## Iteration 200: error is 38.191957 (50 iterations in 0.01 seconds)
## Iteration 250: error is 38.688498 (50 iterations in 0.01 seconds)
## Iteration 300: error is 0.508018 (50 iterations in 0.01 seconds)
## Iteration 350: error is 0.479170 (50 iterations in 0.01 seconds)
## Iteration 400: error is 0.471636 (50 iterations in 0.01 seconds)
## Iteration 450: error is 0.468366 (50 iterations in 0.01 seconds)
## Iteration 500: error is 0.466743 (50 iterations in 0.01 seconds)
## Iteration 550: error is 0.465019 (50 iterations in 0.01 seconds)
## Iteration 600: error is 0.462786 (50 iterations in 0.01 seconds)
## Iteration 650: error is 0.461227 (50 iterations in 0.01 seconds)
## Iteration 700: error is 0.458451 (50 iterations in 0.01 seconds)
## Iteration 750: error is 0.457297 (50 iterations in 0.01 seconds)
## Iteration 800: error is 0.456732 (50 iterations in 0.01 seconds)
## Iteration 850: error is 0.456609 (50 iterations in 0.01 seconds)
## Iteration 900: error is 0.456767 (50 iterations in 0.01 seconds)
## Iteration 950: error is 0.456696 (50 iterations in 0.01 seconds)
## Iteration 1000: error is 0.456468 (50 iterations in 0.01 seconds)
## Fitting performed in 0.19 seconds.
```

Comentarios para el ajuste:

- theta -> aproximación de Barnes-Hut; 0 = exacto (lento), 0.5 = buen compromiso
- pca -> el paper original recomienda una reducción inicial
- pca_scale -> si se escalan los datos previamente, no hace falta
- check_duplicates -> lo mismo, si se hace previamente, no hace falta repetir

6. Construir data frames para graficar

```
construir_df = function(resultado, etiquetas, perplejidad) {
  data.frame(Dim1 = resultado$Y[, 1],
             Dim2 = resultado$Y[, 2],
             Grupo = etiquetas,
             Perplejidad = paste("Perplejidad =", perplejidad))
}

df_completo = do.call(rbind, lapply(names(resultados), function(p) {
  construir_df(resultados[[p]], etiquetas, p)
})))
```

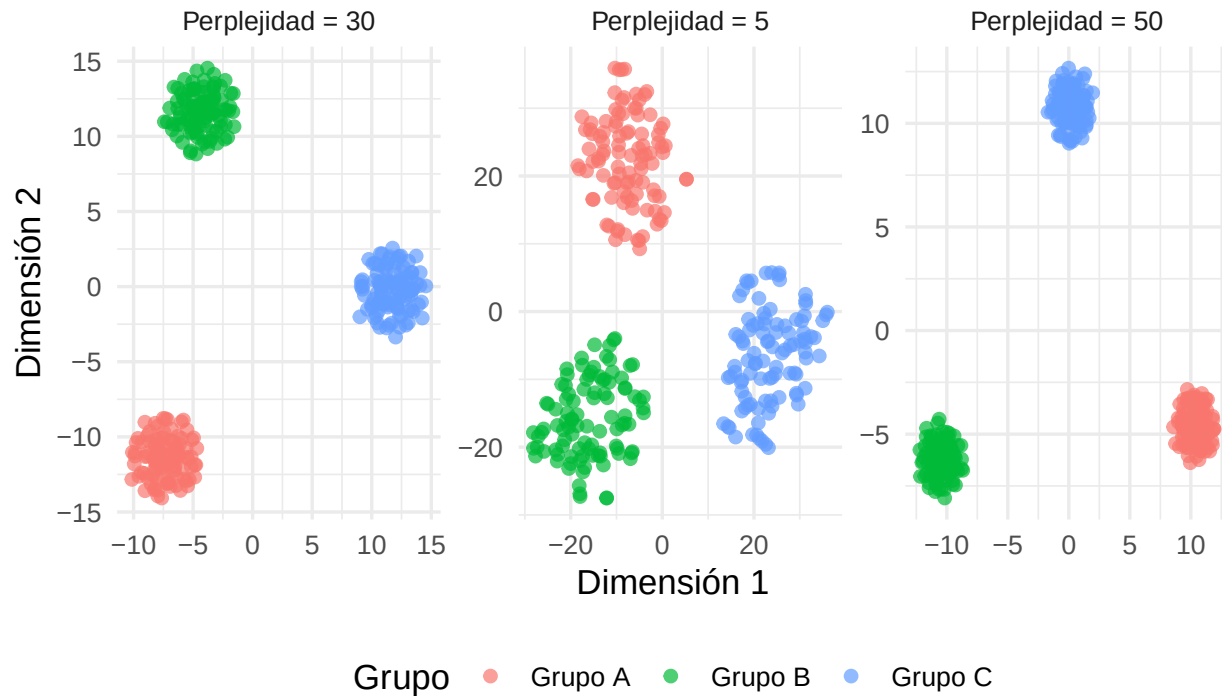
7. Visualización

```
ggplot(df_completo, aes(x = Dim1, y = Dim2, color = Grupo)) +
  geom_point(size = 1.8, alpha = 0.7) +
  facet_wrap(~ Perplejidad, scales = "free") +
  theme_minimal(base_size = 13) +
  labs(title = "t-SNE: efecto de la perplejidad sobre la separación de clústeres",
       subtitle = "Mismos datos, distinta perplejidad",
```

```
x = "Dimensión 1", y = "Dimensión 2") +
theme(legend.position = "bottom")
```

t-SNE: efecto de la perplejidad sobre la separación de clúst

Mismos datos, distinta perplejidad



8. Notas de buenas prácticas (a modo de chuleta):

- Las distancias ENTRE clústeres en el plot no significan nada, solo importa qué cae dentro de cada uno
- Si necesitáis reproducibilidad exacta, documentad la semilla Y la versión del paquete Rtsne (la implementación ha cambiado entre versiones)
- Probad siempre más de un valor de perplejidad antes de quedaros con una conclusión visual; un único gráfico puede engañar
- t-SNE es para explorar, no para inferir. No uséis las coordenadas resultantes como input de un modelo predictivo “porque ya están en 2D”